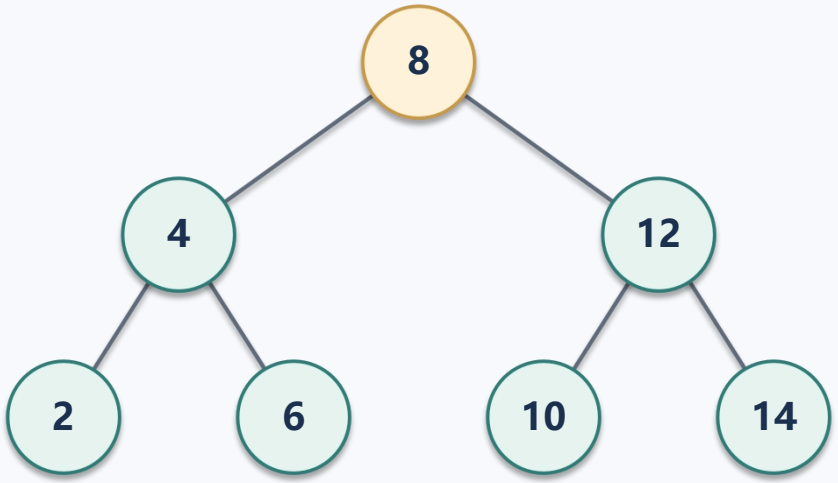
多项式的渐近 与 α-平衡树



汇报人：叶文浩 周锶洋
组　员：陈锦烨 蒋玉佳 叶文浩
余馨兰 周锶洋 周欣怡

多项式的渐近行为

用渐近符号的严格定义，证明五种增长关系

$$p(n) = a_d n^{\mathrm{d}} + a_{d-1} n^{d-1} + \ldots + a_1 n + a_0$$



题设与五个结论

已知 p(n) 是 d 次多项式，最高次项系数 a_d > 0，k 为常量。

$$p(n) = \sum_{i=0}^{d} a_i n^i，且\ a_d > 0$$

a | k ≥ d

$p(n) = O(n^k)$

b | k ≤ d

$p(n) = \Omega(n^k)$

c | k = d

$p(n) = \Theta(n^d)$

d | k > d

$p(n) = o(n^k)$

e | k < d

$p(n) = \omega(n^k)$

共同策略：从定义出发，明确选择常数 c 与阈值 n_0 。

五种渐近记号：只记住量词差别

f、g 最终非负；以下不等式需对所有 $n \geq n_0$ 成立。

记号	常数条件	最终不等式	直观含义
$O(g)$	$\exists\ c>0, n_0$	$f(n) \leq c\, g(n)$	渐近上界
$\Omega(g)$	$\exists\ c>0, n_0$	$c\, g(n) \leq f(n)$	渐近下界
$\Theta(g)$	同时满足 O 与 Ω	$c_1 g(n) \leq f(n) \leq c_2 g(n)$	紧确界
$o(g)$	$\forall\ c>0$，$\exists\ n_0$	$f(n) < c\, g(n)$	严格更慢
$\omega(g)$	$\forall\ c>0$，$\exists\ n_0$	$c\, g(n) < f(n)$	严格更快

重点：O、Ω 只需找到一个常数；o、ω 必须适用于任意给定的正数 c。

若 k ≥ d，则 p(n) = O(n^k)

大 O：找一个 c>0 与 n_0 ，使 0 ≤ p(n) ≤ c n^k。

$$p(n) = a_d n^d + a_{d-1} n^{d-1} + \ldots + a_0$$

取$n \geq 1$，此时对所有$i \leq d$，$n^i \leq n^d \leq n^k$（因为$k \geq d$）。

令$A = \sum_{i=0}^{d} |a_i|$，也就是所有系数绝对值之和

$$|p(n)| \leq |a_d| n^d + |a_{d-1}| n^{d-1} + \ldots + |a_0| \leq \sum_{i=0}^{d} |a_i| n^k = A \cdot n^k$$

取$c = A$，$n_0 = 1$。

当$n \geq n_0$，$0 \leq p(n) \leq c \cdot n^k$，满足大$O$定义。所以$p(n) = O(n^k)$。

若 k ≤ d，则 p(n) = Ω(n^k)

我们把式子提取 n^k：

$$p(n) = n^k(a_d n^{(d-k)} + \sum(i=0)^{(d-1)} a_i n^{(i-k)}) \qquad d-k \geq 0，\ a_d > 0。$$

当 n 足够大时，最高项会远大于后面所有低次项的绝对值。我们取 n_0 足够大，使得：

$$\sum(i=0)^{(d-1)} |a_i| n^{(i-k)} \leq (1/2) a_d n^{(d-k)}$$

于是：

$$p(n) \geq n^k(a_d n^{(d-k)} - (1/2) a_d n^{(d-k)})$$

$$p(n) \geq (1/2) a_d n^d$$

又因为 k≤d，所以 $n^d \geq n^k$。

因此：

$$p(n) \geq (1/2) a_d n^k$$

取$c = (1/2)a_d > 0$，那么当$n \geq n_0$时，$p(n) \geq c \cdot n^k$，满足Ω定义。

因此$p(n) = \Omega(n^k)$。

若 k = d，则 p(n) = Θ(n^d)

根据(Θ)记号定义，只要同时满足O和Ω。

从a：$k = d \geq d$，得$p(n) = O(n^d)$。

从b：$k = d \leq d$，得$p(n) = \Omega(n^d)$。

两者同时成立，所以$p(n) = \Theta(n^d)$。

若 k > d，则 p(n) = o(n^k)

小 o 的定义：

任意$c > 0$，总能找到n_0，$n \geq n_0$时，$p(n) < cn^k$。

考察极限：

$$\lim_{n\to\infty} \frac{p(n)}{n^k} = \lim_{n\to\infty} \frac{a_d n^d + \ldots + a_0}{n^k} = 0$$

因为 k > d，分母阶数更高，极限等于 0。极限为 0 等价于小 o。

也可以放缩证明：

$$\frac{p(n)}{n^k} \leq \frac{An^d}{n^k} = A \cdot n^{d-k}$$

d-k < 0，$n \to \infty$时，An^{d-k} 可以小于任意给定的 c。

对任意$c > 0$，选足够大n_0，$n \geq n_0$，$p(n) < cn^k$。满足$o(n^k)$定义。

若 k < d，则 p(n) = ω(n^k)

ω定义：对任意$c > 0$，存在n_0，当$n \geq n_0$，$p(n) > cn^k$。

同样看极限：

$$\lim_{n\to\infty} \frac{p(n)}{n^k} = \lim_{n\to\infty} \frac{a_d n^d + \ldots}{n^k} = +\infty$$

d > k，分子增长更快，极限无穷大。

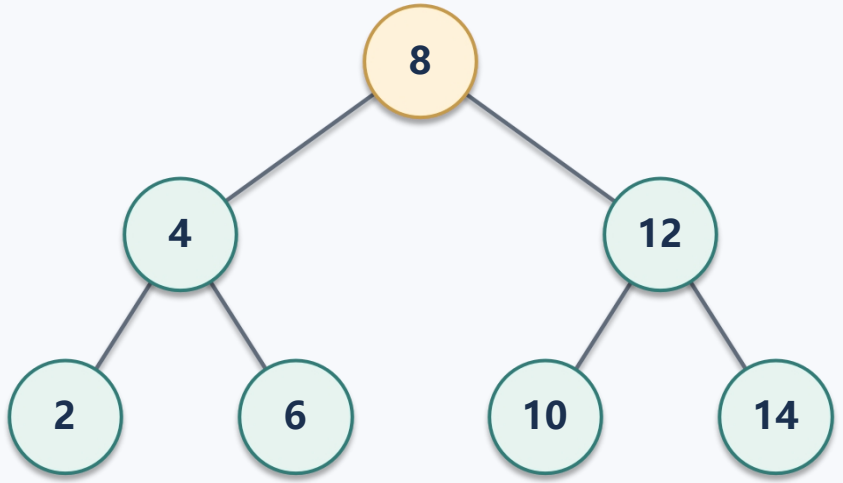
我们简单放缩：

$$\frac{p(n)}{n^k} = a_d n^{d-k} + \text{低次项}$$

d-k > 0，n 趋向无穷时，这个式子可以超过任意给定常数 c_0。

对任意$c > 0$，总能找到n_0，当$n \geq n_0$，$p(n) > c \cdot n^k$，满足ω定义。

摊还加权平衡树



普通二叉搜索树的问题

BST 的关键字次序正确，不代表它的形状足够平衡。

顺序插入： 1, 2, 3, 4, 5, 6



退化成链

树高 $h = \Theta(n)$，搜索可能访问全部节点。

理想情况

树高 $h = \Theta(\lg n)$，搜索只访问对数级节点。

目标不是每次都做昂贵的全局平衡，而是允许局部失衡，并在必要时重建子树。

维护机制：插入 / 删除 → 检查平衡 → 必要时重建。

摊还加权平衡树

16-3 （摊还加权平衡树） 考虑扩充普通二叉搜索树，为每个节点 x 增加属性 $x.size$，此属性给出了根为 x 的子树中关键字的数量。令 α 为满足 $1/2 \leqslant \alpha < 1$ 的一个常数。对于一个给定节点 x，当 $x.left.size \leqslant \alpha \cdot x.size$ 且 $x.right.size \leqslant \alpha \cdot x.size$ 时，我们称 x 是 **α- 平衡的**。如果树中每个节点都是 α - 平衡的，则称树整体是 α - 平衡的。G.Varghese 提出了如下摊还方法来维护加权平衡树。

a. 在某种意义上，一棵 1/2- 平衡树达到了极限的平衡。给定任意一棵二叉搜索树中的一个节点 x，说明如何重建以 x 为根的子树，使得它变为 1/2- 平衡的。你的算法的运行时间应该为 $\Theta(x.size)$，可以使用 $O(x.size)$ 的辅助空间。

b. 证明：在一棵 n 个节点的 α - 平衡二叉搜索树中执行一次搜索操作的最坏情况时间为 $O(\lg n)$。

01

考虑扩充普通二叉搜索树:

为每个节点 x 增加属性 x.size ， 此属性给出了根为 x 的子树中关键字的数量。

02

令 α 为满足 $1/2 \leqslant \alpha < 1$的一个常数。对于一个给定节点 x :

当 $x.left.size \leqslant \alpha \cdot x.size$ 且 $x.right.size \leqslant \alpha \cdot x.size$ 时

我们称 x 是 $\alpha -$ 平衡的。

03

如果树中每个节点都是 $\alpha -$ 平衡的，则称树整体是 $\alpha -$ 平衡的。

回答什么？

围绕题目 a–e，建立一条完整的证明链。

a | 如何重建 1/2 平衡树？

将 BST 中序遍历成有序数组，再递归取中位数作为根。

b | 搜索的时间复杂度？

每向下一层，子树规模最多缩小为原来的 α 倍。

c–d | 势能怎样支付？

失衡积累势能；重建后子树内部势能归零。

e | 更新的摊还代价？

普通更新走一条对数高度的路径，重建由势能支付。

核心思想：不要求每一步都便宜，而是控制一串操作的平均成本。

子树规模与 α-平衡

空子树的 size 记为 0。

子树规模

$$x.size = 1 + x.left.size + x.right.size$$

左右子树约束

$$x.left.size \le \alpha \cdot x.size$$
$$x.right.size \le \alpha \cdot x.size$$

参数范围： $1/2 \le \alpha < 1$

α 是固定常数。α 越接近 1，允许的局部失衡越大；但只要 α < 1，树高仍可由对数界控制。

α = 1/2

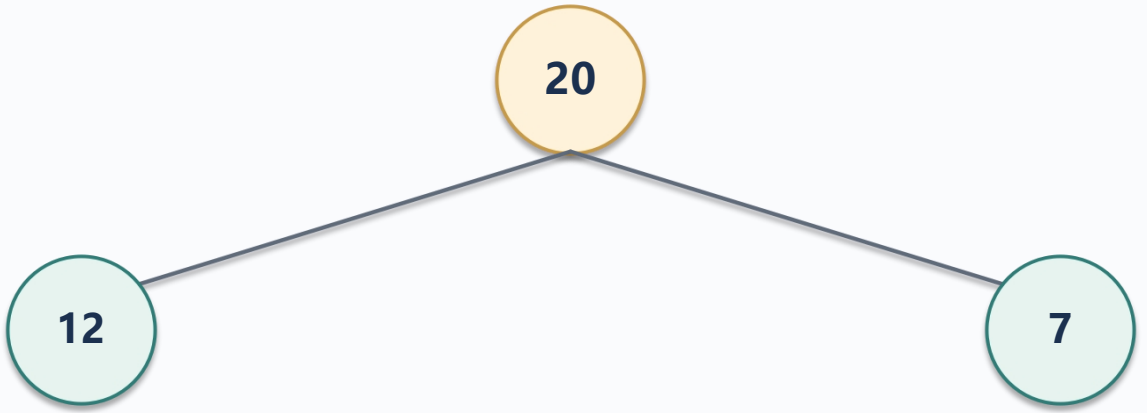
左右子树规模都不超过总规模的一半。

α > 1/2

留出“失衡缓冲区”，适合摊还维护。

用规模理解 α-平衡

例如 α = 0.7，根的子树规模为 20。



$12 \le 0.7 \times 20 = 14$

$7 \le 0.7 \times 20 = 14$

节点数满足比例约束；这不是在直接限制左右子树的高度。

注：左右子树规模之和为 19，因为根节点自身占 1 个节点。

重建的三步法

把“任意形状的 BST”变成“规模尽量均匀的 BST”。

步骤 1 | 遍历

对目标子树做中序遍历，得到递增序列：

$a_1 < a_2 < \dots < a_m$

步骤 2 | 选中位数

取中间关键字作为根；中位数左边递归构建左子树，右边递归构建右子树。

步骤 3 | 连接

将新建的左右子树接到根上，并将重建子树接回原树。

关键点：重建保持中序次序，所以不会破坏二叉搜索树性质。

设目标子树有 m 个节点：遍历 $\Theta(m)$，构建 $\Theta(m)$，总时间 $\Theta(m)$。

递归构造：中位数作为根

示例：有序数组 [1,2,3,4,5,6,7]。



递归终止条件：当前区间为空，返回空子树。

每个节点恰好被选作一次根，因此构建工作总量与节点数成正比。

分治重建伪代码

```
rebuild ( nodes , low , high )
    if( low>high )     return ;
    mid = floor ( ( low + high ) / 2 ) ;
    root = nodes [ mid ] ;
    root.left = rebuild ( nodes , low , mid-1 ) ;
    root.right = rebuild ( nodes , mid+1 , high ) ;
    root.size = root.left.size + root.right.size ;
    return root ;
```

证明重建后是 1/2-平衡树

设当前递归处理的子树有 k 个节点。

中位数被选为根，左右子树规模为：

左子树

$\lfloor (k-1)/2 \rfloor$

右子树

$\lceil (k-1)/2 \rceil$

两边规模均 ≤ k/2，因此满足左右子树规模 ≤ 1/2 • k。

递归子树也采用相同构造，所以每个节点都满足 1/2-平衡条件。

重建算法的复杂度

令 m = x.size。

中序遍历

每个节点访问一次：Θ(m)

数组 / 节点引用

保存序列：O(m) 空间

递归建树

每个节点处理一次：Θ(m)

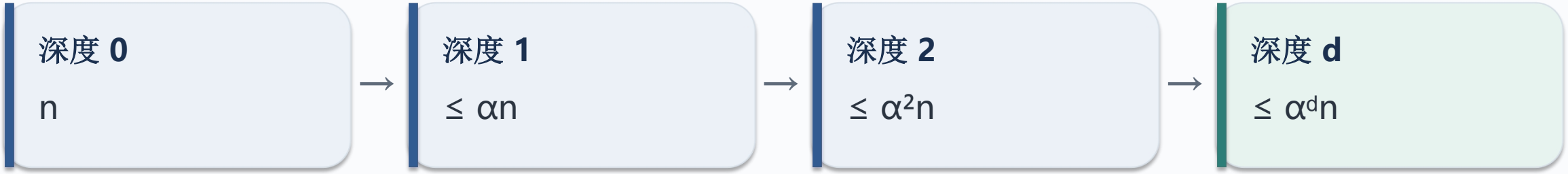
总时间： Θ(m) + Θ(m) = Θ(m)

辅助空间： O(m)（数组与递归调用栈的上界）。

从局部平衡推导树高

平衡条件限制每一层能保留多少节点。

根的子树规模为 n；每往下一层，子树规模至多乘以 α。



若深度 d 的节点存在，则其子树至少含 1 个节点： $1 \le n\alpha^d$。

于是 $d \le \lg n / \lg(1/\alpha) = O(\lg n)$，因为 α 是固定常数且 $\alpha < 1$。

对数推导：注意不等号方向

关键是 $\ln \alpha < 0$。

从 $1 \le n\alpha^d$ 出发：

第 1 步 $\alpha^d \ge 1/n$

第 2 步 $d \cdot \lg \alpha \ge -\lg n$

第 3 步 $d \le (\lg n) / \lg (1/\alpha)$

搜索沿一条根到叶的路径进行，因此时间与树高同阶。

摊还加权平衡树

对于本问题的剩余部分，假定常数 α 严格大于 1/2。假定你实现的 INSERT 和 DELETE 算法与普通 n 节点二叉搜索树的算法是一样的，差别仅在于，如果发现树中任何节点不再是 α - 平衡的，则在最高的不平衡节点，对以它为根的子树执行“重建”，使其变为 1/2 平衡的。

我们用势能法分析此重建方法。对于二叉搜索树 T 中的一个节点 x，定义

$$\Delta(x) = |x.left.size - x.right.size|$$

定义 T 的势函数为

$$\Phi(T) = c \sum_{x \in T:\Delta(x) \geqslant 2} \Delta(x)$$

其中 c 是一个足够大的常数，它依赖于 α。

c. 证明：任意二叉搜索树的势都是非负的，1/2- 平衡树的势为 0。

d. 假定 m 个单位的势足够支付重建 m 个节点子树的代价。相对于 α 来说，c 应该多大才能使得重建一棵非 α - 平衡子树的摊还时间为 $O(1)$。

e. 证明：在一棵 n 个节点的 α - 平衡树中插入或删除一个节点的摊还时间为 $O(\lg n)$。

定义失衡量 Δ(x)

势函数只为“明显失衡”的节点记账。

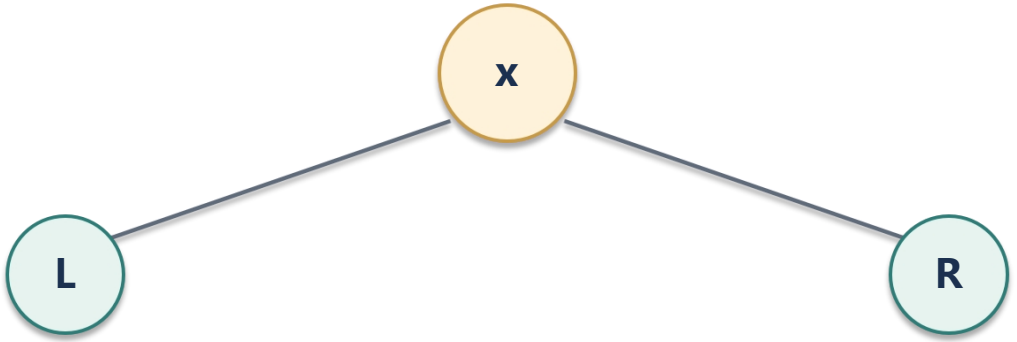
节点失衡量

$\Delta(x) = |x.left.size - x.right.size|$

整棵树的势

$\Phi(T) = c \sum \Delta(x)$，只对 $\Delta(x) \geq 2$ 求和

$c > 0$ 是待选常数；它决定每单位失衡对应多少势能。



左子树规模

右子树规模

证明：任意树的势非负

势能是非负量的加权和。

对任意节点 x：$\Delta(x) = |x.left.size - x.right.size| \geq 0$。

势函数只累加满足 $\Delta(x) \geq 2$ 的项，且 $c > 0$。

因此

$$\Phi(T) \geq 0$$

这保证势能不会成为“负债”：累计摊还分析可以使用势能差。

证明：1/2-平衡树的势为 0

先从 size 恒等式推导左右规模差。

$$x.size = 1 + x.left.size + x.right.size$$

1/2-平衡意味着：$x.left.size \le x.size/2$，$x.right.size \le x.size/2$。

由 $x.left.size \le x.size/2$ 知，$x.left.size \le (1 + x.left.size + x.right.size)/2$

化简得 $x.left.size - x.right.size \le 1$；

同理有 $x.right.size - x.left.size \le 1$；

所以左右子树规模之差至多为 1：$\Delta(x) \le 1$。

求和条件 $\Delta(x) \ge 2$ 不成立

没有任何节点贡献势能，因此 $\Phi(T) = 0$。

摊还成本的基本公式

实际成本与势能变化一起计算。

m个单位的势能支付重建m个节点子树的代价：
对于 α ， c 应多大才能使重建 非 α-平衡树 的 摊还时间为O（1）

摊还成本

$\hat{C}$ = 实际成本 + 势能变化（后势能-初势能）

实际成本：Θ（m）

👉 求要积攒多少势能 $\Phi(T) = c \sum \Delta(x)$

注意：这是摊还意义上的成本，不是单次实际运行时间。

摊还成本

① 已知重建 实际成本 $\Theta(m)$ 设$k \cdot m$，$k > 0$

② 分析势能变化

由C，重建后势能为 0

$\Delta\Phi = 0 - \Phi(before) = -\Phi(before)$

③ 为什么 x 会触发重建

x 是最高不平衡节点（祖先都平衡）

假设左子树更重 $L > \alpha \cdot m$

而 m = L + R + 1 👉 R = m − L − 1

则 $\Delta(x) = L - R = L - (m - L - 1) = 2L - m + 1$

因为 $L > \alpha \cdot m$ 则 $\Delta(x) > (2\alpha - 1)m + 1$

则 $C \cdot \Delta(x) \geq c(2\alpha - 1)m$

即 $\Phi_{(before)} \geq c(2\alpha - 1)m$

$\Delta\Phi \leq -c(2\alpha - 1)m$

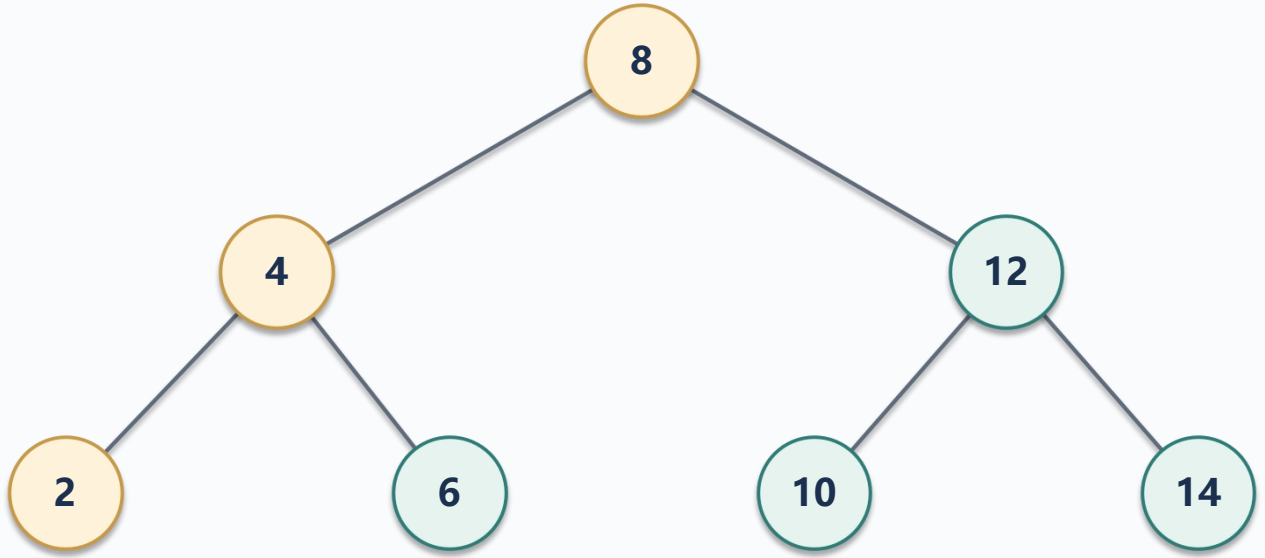
④ 摊还代价 $\leq k \cdot m - c(2\alpha - 1)m = [k - c(2\alpha - 1)]m$

又O(1) 则与 m 无关 则 $k - c(2\alpha - 1) \leq 0$

解得 $c \geq \frac{k}{2\alpha - 1}$

普通更新只影响一条搜索路径

从根到插入位置 / 删除位置，路径长度为 O(lg n)。



只有路径上的 O(lg n) 个节点，其子树规模可能发生变化。

每个受影响节点的 Δ 最多改变 1。

普通更新引起的势能变化

势函数只在 Δ 跨过阈值 2 时开始 / 停止计账。

受影响节点数

$O(\lg n)$

单节点势能变化

至多 $O(c)$

总变化上界

$O(c \lg n)$

因此普通插入 / 删除的摊还成本为 $O(\lg n)$（c 为固定常数）。

如果随后触发重建，**d 问已证明重建的摊还成本为 $O(1)$。**

合并：总摊还时间为 O(lg n)

普通更新与重建分别记账。

普通更新

O(lg n)

+

重建

O(1)

=

总摊还成本

O(lg n)

结论：插入或删除一个节点的摊还时间是 O(lg n)。

这不是说每次操作的实际时间都为对数级；某一次重建仍可能花 **Θ(n)**。

谢谢聆听

Questions & Discussion

